

1 - classical crypto

Performed by *Nikita Niakhai*

Teammate is *Bogdan Vilyouga*

Nikita was responsible for creating 2 plaintexts, encrypting and decrypting using 2 ciphers such as Vigenere and Nihilist. Nikita has made a current report that represents his pipeline.

Description of tasks:

1. 50 words and then exchange it with bro, he cracks my text, and I crack his with Vigenere cipher text
2. then use a random cipher and exchange with bro and try cracking

TERMS [0]:

Confusion - (from the verb "confused"), this concept is one of the keys to understanding symmetric cipher design. Simply, it obscures the connection/link between plain and cipher texts (pretends that there is no connection between them)

Diffusion - almost opposite to confusion, means that we disperse plaintext statistics in the ciphertext. Metaphorically, we sow small seeds of plaintext in the ciphertext.

Permutation - means replacing parts of plaintext with each other in different sizes: it may be a simple interchange of all letters in the sentence. Characters of all words from the message stay the same, but their position changes.

One of the famous applications appeared in the Enigma machine, which was invented by the Nazis, and was cracked by a Polish cryptographer for Britain. This machine needed a key to configure a system (set a proper permutation).

Substitution - reflects programming concepts of the "map" function, dictionary. E.g., the first step to encrypt a message with the Vigenere cipher is to map all English letters to their byte analogue.

Substitution comes in different forms. We map letters with numbers (s), we can map words to other words.

Substitution technique was often used in the XX century, for example, there were the famous Zimmerman telegram, encrypted with codebook - is a simple dictionary of words that mapped to 5-digits numbers: "February" corresponded to 13605. To decrypt the ciphertext, people used the codebook of digits, first hand.

Additive. Modern ciphers use many codebooks at once, they point to each other. And to communicate parts send AKA "additive", a simple message mostly first characters of ciphertext, that gives a hint where to look up in codebook, or which one to use. Theoretically,

it is a good concept, but practically, people reused the same additive many times that led to flaws in cipher design and eventually to cracking.

****TASK 1 (my texts)**

I used <https://www.dcode.fr/>

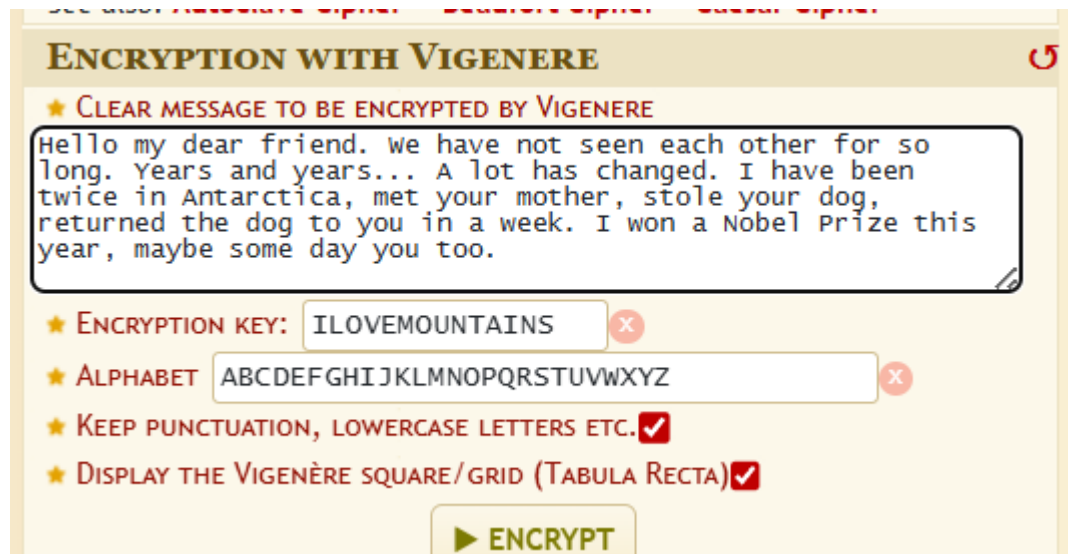
key (14 letters) : ilovemountains

Plain text (52 words):

Hello my dear friend. We have not seen each other for so long. Years and years... A lot has changed. I have been twice in Antarctica, met your mother, stole your dog, returned the dog to you in a week. I won a Nobel Prize this year, maybe some day you too.

Cipher text :

Ppzgs ym xrtr neamyr. Ri topr gob fwmy svgt cnuxr nbj az zjrs. Mynks iav gpomw... M zig aaa pziyuzh. U vuix bmrh bhwxi ub Uamazplqno, hif mihk mwgzmc, gosxs sbnr lby, zphpvzsx gae lby bz mgy ub u jxes. V owy o lsnsf Ckihr lptg timf, gnrbm fgup rvc kco gho.



The screenshot shows a web interface titled "ENCRYPTION WITH VIGENERE". It contains a text area with the plain text: "Hello my dear friend. we have not seen each other for so long. Years and years... A lot has changed. I have been twice in Antarctica, met your mother, stole your dog, returned the dog to you in a week. I won a Nobel Prize this year, maybe some day you too." Below the text area, there are input fields for the "ENCRIPTION KEY:" (ILOVEMOUNTAINS) and "ALPHABET" (ABCDEFGHIJKLMNOPQRSTUVWXYZ). There are also checkboxes for "KEEP PUNCTUATION, LOWERCASE LETTERS ETC." (checked) and "DISPLAY THE VIGENÈRE SQUARE/GRID (TABULA RECTA)" (checked). An "ENCRYPT" button is at the bottom.

****TASK 1 (teammates' ciphertext)**

I used an online tool at guballa.de to crack it.

Cipher text :

Bfueq qk mpf qmzjk sx Awibxfmi. Klaa al bip hip klw mvnkbemwe rx Avfhxpwma Lemnmjlquj wbrixk, ifw ubyc vvn xzqfza dzqm. Zk mk ifgwztro kyel 50 egkl sedv ks tm okqueiv svgsckx Q bx vceemfo gnb pq murxmfilbwo es kfdi mx obbi elmjv. Nmal t tjextv sml ugkm. Byh nzeedtq!

****Found key (14 letters)** : irresistible

Cracked plain text:

today is the first of september this is the day the education at innopolis university starts and

many new things come it is annoying that words have to be written because i am running out of imagination to come up with these just a little bit more and finally

Input

Cipher Text *

Bfueq qk mpf qmzjk sx Awibxfmi. Klaa al bip hip klw mvnkbemwe rx Avfhxpwna Lemnnjlquj
wbrixk, ifw ubyc vvm xzqfza dzqm. Zk mk ifgwztro kyel SØ egklt sedv ks tm okqueiv svgsckx
Q bx vceemfo gnb pq murxmfilbwo es kfdi mx obbi elnjv. Nmal t tjextv sml ugkm. Byh
nzeedtq!

Cipher Variant

Classical Vigenere

Language

English

Key Length*

3-30

Break Cipher Clear Cipher Text

Result

Cleartext using the keyword "irresistible":

Today is the first of September. This is the day the education at Innopolis University starts, and many new things come. It is annoying that 50 words have to be written because I am running out of imagination to come up with these. Just a little bit more. And finally!

Details

Key	"irresistible"
Key length	12
Ciphertext length	211
Ratio (cipher_len/key_len)	17.58
Difficulty	easy
Clear text score (fitness)	101.57

Clear text score

50 80 100

TASK 2 (my texts)

The Nihilist cipher is an over-encryption of the Polybius square.

Plain text (62 words, 335 chars):

Sometimes I feel very veird: everything seems to be so unreal, vords become longer, stretching; lines I see become nonlinear; fog comes into my vief. Yesterday a guy that vas shoot on Vederessa St. He vas so thin, he reminded me about my first case in this bureau. Almost 2 decades and I am almost done. My body and mind are exhausted.

key (normal alphabet without w) : ABCDEFGHIJKLMNOPQRSTUVWXYZ

Cipher text:

78 59 56 39 77 48 77 60 78 48 44 39 47 56 96 60 77 78 75 39 56 67 58 60 86 39 66 78 77
 47 68 79 56 68 38 39 65 68 89 80 46 39 67 59 83 58 87 60 45 56 75 59 75 38 88 57 49 37
 58 57 47 56 79 79 56 39 66 68 77 67 59 90 47 47 47 58 54 56 68 79 49 68 47 68 47 39 56
 60 47 59 56 39 66 59 78 77 58 58 38 35 75 45 79 67 47 59 56 39 76 48 78 90 69 57 77 76
 56 39 96 99 49 68 68 39 75 38 55 99 45 46 74 78 77 47 55 90 86 35 67 68 55 59 79 90 69
 58 75 39 46 39 87 60 78 68 34 68 77 47 59 97 45 68 67 59 77 47 68 79 57 39 66 39 65 48
 78 59 49 38 56 39 43 36 79 96 79 57 77 45 56 67 88 90 47 35 67 39 56 58 89 68 58 68 35
 75 75 39 55 96 45 56 56 59 76 69 58 60 47 35 37 39 76 35 78 59 58 35 56 35 64 57 79 89
 79 38 58 58 47 57 98 57 69 38 77 35 66 38 77 69 68 38 34 67 47 39 97 68 45 75 67 69 47
 38

your feedback on the new page: Nihilist Cipher!

NIHILIST CIPHER

Cryptography › Poly-Alphabetic Cipher › Nihilist Cipher

NIHILIST CIPHER DECODER

★ NIHILISTS CIPHERTEXT

78	59	56	39	77	48	77	60	78	48	44	39	47	56	96	60	77	78	75
39	56	67	58	60	86	39	66	78	77	47	68	79	56	68	38	39	65	68
89	80	46	39	67	59	83	58	87	60	45	56	75	59	75	38	88	57	49
37	58	57	47	56	79	79	56	39	66	68	77	67	59	90	47	47	47	58
54	56	68	79	49	68	47	68	47	39	56	60	47	59	56	39	66	59	78
77	58	58	38	35	75	45	79	67	47	59	56	39	76	48	78	90	69	57

★ ENCRYPTION GRID

\	1	2	3	4	5
1	A	B	C	D	E
2	F	G	H	I	J
3	K	L	M	N	O
4	P	Q	R	S	T
5	U	V	X	Y	Z

5 x 5 RESIZE

CLEAR

L ABCDEFGHIJKLMNOPQRSTUVWXYZ

★ COORDINATES ☒ Row-COLUMN (DEFAULT)
☐ COLUMN-Row

TASK 2 (teammates' ciphertext)

Input:

Received a cipher text:

3732695931686518661968121854738248669251532811243881686168651866581327438
 8146268322133857561331625673925712681848321689112181632748639125184956265
 3956274211743881625218422575361631686132567327162524388133175712581848813
 8914373586617121633624281596919181484953121956216252473833571961671376856

9516732437359673714839249385186518668661738386313126855126658685475712149
5626196856912698593

Divided text into 2-digits pairs:

37 32 69 59 31 68 65 18 66 19 68 12 18 54 73 82 48 66 92 51 53 28 11 24 38 81 68 61 68
65 18 66 58 13 27 43 88 14 62 68 32 21 33 85 75 61 33 16 25 67 39 25 71 26 81 84 83 21
68 91 12 18 16 32 74 86 39 12 51 84 95 62 65 39 56 27 42 11 74 38 81 62 52 18 42 25 75
36 16 31 68 61 32 56 73 27 16 25 24 38 81 33 17 57 12 58 18 48 81 38 91 43 73 58 66 17
12 16 33 62 42 81 59 69 19 18 14 84 95 31 21 95 62 16 25 24 73 83 35 71 96 16 71 37 68
56 95 16 73 24 37 35 96 73 71 48 39 24 93 85 18 65 18 66 86 61 73 83 86 31 31 26 85 51
26 65 86 85 47 57 12 14 95 62 61 96 85 69 12 69 85 93

192 words

Divided text into 3-digits pairs:

373 269 593 168 651 866 196 812 185 473 824 866 925 153 281 124 388 168 616 865 186
658 132 743 881 462 683 221 338 575 613 316 256 739 257 126 818 483 216 891 121 816
327 486 391 251 849 562 653 956 274 211 743 881 625 218 422 575 361 631 686 132 567
327 162 524 388 133 175 712 581 848 813 891 437 358 661 712 163 362 428 159 691 918
148 495 312 195 621 625 247 383 357 196 167 137 685 695 167 324 373 596 737 148 392
493 851 865 186 686 617 383 863 131 268 551 266 586 854 757 121 495 626 196 856 912
698 593

128 words

Divided text into 4-digits pairs:

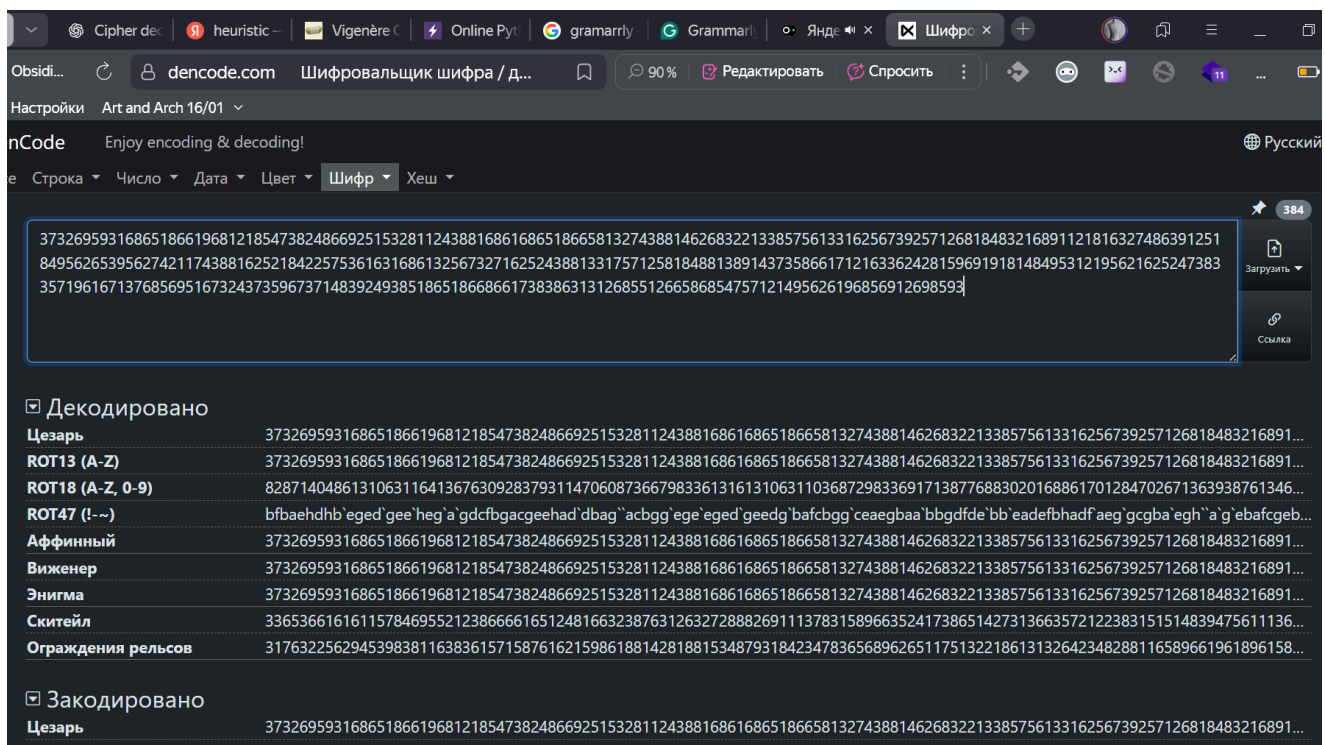
3732 6959 3168 6518 6619 6812 1854 7382 4866 9251 5328 1124 3881 6861 6865 1866
5813 2743 8814 6268 3221 3385 7561 3316 2567 3925 7126 8184 8321 6891 1218 1632
7486 3912 5184 9562 6539 5627 4211 7438 8162 5218 4225 7536 1631 6861 3256 7327
1625 2438 8133 1757 1258 1848 8138 9143 7358 6617 1216 3362 4281 5969 1918 1484
9531 2195 6216 2524 7383 3571 9616 7137 6856 9516 7324 3735 9673 7148 3924 9385
1865 1866 8661 7383 8631 3126 8551 2665 8685 4757 1214 9562 6196 8569 1269 8593
96 words

Steps

1. Visual analysis. I see repeated 121 combination. The text contains 384 characters (as shown in Notepad app). I see that randomly 4 times 121 combination appears, probably might be a '.' symbol, meaning that this is the end of the sentence stays for. But this might go with every other combination, so I stop thinking this way.
2. I try to determine a cipher. Found some web-sites that have analyzed the ciphertext [3], [4]. Maybe this is a decimal cipher, maybe this is a tridigital cipher.
3. I try different ASCII decoders with simple variants of the integer where i do split after each 2 and 3 letter [CodeSnippet1]. But no result. Actually, it is not ASCII there would be

more "1" digits.

4. Found another decoder site that shows different encodes/decodes if you put text in the input [5]



5. I decide to do the freq. analyses on 2 and 3-digits pairs, in order to see the relation to alphabet frequency. 3-digits pairs - nothing interesting, 2-digits too [FreqAn1], [FreqAn2]. Also, I did for 4-digits, but only 4 numbers occur 2 times, others only one.
6. I am thinking that I need to change the methods completely, because, If I knew a cipher name and even a key, it does not mean that there is no depth of cipher, so I need to check all different combinations of it. CTF smokes nearby...
7. Probably there is a chance that my teammate has created a codebook for letters/words that corresponds to some weird number
8. I tried to put a cipher on cryptii.com on every cipher without changing the key - nothing. *I will not put 20+ screenshots of nothing to show this pain;)*
9. Idea comes to try using Substitution: every number is for a letter of the alphabet. But there might be cases of 1x and 2x numbers... Smells like hard brute force with computing powers
 1. But there is assumptions from the book that a cryptosystem is assumed secure if there is no known shortcut attack: the best known attack is equally resources-intensive and efforts to exhaustive key search.
10. I try to ask GPT [6] what he thinks about my cipher. He just analyzed probable types of cipher, the same as I mentioned above. He suggested converting between different codes, from HEX to ASCII (i used online converters here[7]).... Also he himself did some stupid brute force and did not show me the code for it.
11. I am thinking asking him merely just for methodology and steps. Maybe some new useful ideas will come.

12. New day. I ask GPT tools to help me 1) find out the name of the cipher 2) exhaustive key search for a specific cipher system.

I need to decode/crack this ciphertext and find out what the plaintext is. I know that it is ~50 english words and some of known ciphers were used to encrypt it. Only one cipher was used, depth of ciphers is absent. Maybe spaces between numbers were deleted - IDK. Don't try to bruteforce it or crack it. I need from you list of possible tools and online tools that can help me find out, firstly the cipher and then exhaustive key search for this specific cipher

13. Found cipher detector on dcode.fr [8]. Detection showed Morbit Cipher.

The screenshot shows the dcode.fr interface. On the left, there's a search bar with 'e.g. type 'sudoku'' and a 'SEARCH A TOOL ON dCODE' button. Below it, a 'BROWSE THE FULL dCODE TOOLS' LIST' button. The 'results' section shows 'dcode's analyzer suggests to investigate:' with three options: 'Morbit Cipher' (selected), 'Hexadecimal (Base 16)', and 'ASCII Code'. On the right, the 'CIPHERTEXT TO RECOGNIZE' section displays a long string of numbers: 6865186658132743881462683221338575613316256739257126818483216891121816327486391251849562653956274211743881625218422575361631686132567327162524388133175712581848813891437358661712163362428159691918148495312195621625247383357196167137685695167324373596737148392493851865186686617383863131268551266586854757121495626196856912698593. Below this is a 'CLUES/KEYWORDS (IF ANY)' field and an 'ANALYZE' button. At the bottom, it says 'See also: Frequency Analysis - Index of Coincidence', 'SYMBOLS IDENTIFIER', and 'Go to: Symbols Cipher List'.

14. Morbit cipher is a form of Morse code that uses substitution. Firstly, we get a word and then assign to each letter its number in the alphabet, so we get 9-digit key. In Morse code there are 9 bigrams (pairs of morse characters). So possible keys (complexity) are $9! = 362880$.
15. I tried to find an online solution to exhaustive key search, but nothing useful after first 10 links in Google, so I decided to write my own brute force solution in Python. The idea is to try all keys and save all possible plaintext results. Plaintext match is the one that matches the some most common words in English.
16. Code transfers: ciphertext -> possible bigrams -> full morse string -> split with x or xx separator -> map from morse to text
17. Found the result.

1. Numeric 9-digit key: 896273154. Letter analogue key: HIFBGCAED

```
--- Candidate 22 (tested 260633 perms) ---
Numeric 9-digit key: 896273154
< Letter analogue key: HIFBGCAED
Plaintext:
TODAY IS THE FIRST OF SEPTEMBER THIS IS THE DAY THE EDUCATION AT INNOPOLIS UNIVERSITY STARTS AND MANY NEW THINGS COM
E IT IS ANNOYING THAT WORDS HAVE TO BE WRITTEN BECAUSE I AM RUNNING OUT OF IMAGINATION TO COME UP WITH THESE JUST A
LITTLE BIT MORE AND FINALLY

--- Candidate 22 (tested 260633 perms) ---
Numeric key mapping:
1->x. 2->-. 3->-x 4->xx 5->x- 6->..x 7->-- 8->... 9->.-
Alphabetical analogue key:
A->x. B->-. C->-x D->xx E->x- F->..x G->-- H->... I->.-
Plaintext:
TODAY IS THE FIRST OF SEPTEMBER THIS IS THE DAY THE EDUCATION AT INNOPOLIS UNIVERSITY STARTS AND MANY NEW THINGS COM
E IT IS ANNOYING THAT WORDS HAVE TO BE WRITTEN BECAUSE I AM RUNNING OUT OF IMAGINATION TO COME UP WITH THESE JUST A
LITTLE BIT MORE AND FINALLY
<
```

CodeSnippet1 (useless)

```
main.py
1 # Define your big integer
2 ciphertext =
3
4 # Convert to string for splitting
5 s = str(ciphertext)
6
7 # Split into 2-digit groups
8 groups = [s[i:i+3] for i in range(0, len(s), 3)]
9
10 # Print in one line, separated by spaces
11 print(" ".join(groups))
12
```

input

```
373 269 593 168 651 866 196 812 185 473 824 866 925 153 281 124 388 168 616 865 186 658 132 743 881 462 683 221 338
575 613 316 256 739 257 126 818 483 216 891 121 816 327 486 391 251 849 562 653 956 274 211 743 881 625 218 422 575
361 631 686 132 567 327 162 524 388 133 175 712 581 848 813 891 437 358 661 712 163 362 428 159 691 918 148 495 312
195 621 625 247 383 357 196 167 137 685 695 167 324 373 596 737 148 392 493 851 865 186 686 617 383 863 131 268 551
266 586 854 757 121 495 626 196 856 912 698 593
```

CodeSnippet2 (useful)

```
import itertools
import sys

# ----- CONFIG -----
CIPHERTEXT =
"373269593168651866196812185473824866925153281124388168616865186658132743881
4626832213385756133162567392571268184832168911218163274863912518495626539562
7421174388162521842257536163168613256732716252438813317571258184881389143735
8661712163362428159691918148495312195621625247383357196167137685695167324373
5967371483924938518651866866173838631312685512665868547571214956261968569126
98593"
TARGET = "" # heuristic mode

MIN_COMMON_WORDS = 3
COMMON_WORD_RATIO = 0.25
MIN_ALPHA_RATIO = 0.7

# -----

BIGRAMS = ['..', '.-', '.x', '-.', '--', '-x', 'x.', 'x-', 'xx']

MORSE_TO_TEXT = {
    '.-': 'A', '-...': 'B', '-.-.': 'C', '-...': 'D', '.': 'E', '-.-.': 'F', '-
-': 'G', '....': 'H', '...': 'I',
    '.---': 'J', '-.-': 'K', '-...': 'L', '--': 'M', '-.': 'N', '---': 'O', '.-
-': 'P', '--.': 'Q', '.-': 'R',
    '...': 'S', '-': 'T', '-.-': 'U', '...-': 'V', '-.-': 'W', '-.-': 'X', '-.-
': 'Y', '--...': 'Z',
    '-----': '0', '.-----': '1', '-.-.-': '2', '...-': '3', '....-
': '4', '.....': '5', '-....': '6',
    '-...': '7', '----.': '8', '-----': '9'
}
```

```

COMMON_WORDS = {

    "the", "and", "that", "have", "for", "not", "with", "you", "this", "but", "his", "from",
    "they", "say", "she",

    "will", "one", "all", "would", "there", "their", "what", "so", "up", "out", "if", "about",
    "who", "get", "which",

    "go", "me", "when", "make", "can", "like", "time", "no", "just", "him", "know", "take",
    "people", "into", "year",

    "your", "good", "some", "could", "them", "see", "other", "than", "then", "now", "look",
    "only", "come", "its",

    "over", "think", "also", "back", "after", "use", "two", "how", "our", "work", "first",
    "well", "way", "even",
    "new", "want", "because", "any", "these", "give", "day", "most", "us"
}

def digits_to_bigram_string(cipher_digits, mapping):
    return ''.join(mapping[ch] for ch in cipher_digits if ch in mapping)

def morse_to_plain(morse_str):
    words = morse_str.split('xx')
    plain_words = []
    for w in words:
        if not w:
            continue
        letters = w.split('x')
        decoded = []
        for l in letters:
            if not l:
                continue
            decoded.append(MORSE_TO_TEXT.get(l, '?'))
        if decoded:
            plain_words.append(''.join(decoded))
    return ' '.join(plain_words)

def looks_like_english(text):
    if not text:
        return False
    chars = len(text)
    alpha = sum(1 for c in text if c.isalpha())
    if chars == 0 or alpha / chars < MIN_ALPHA_RATIO:
        return False
    words = text.lower().split()
    if not words:
        return False

```

```

        common_count = sum(1 for w in words if w.strip('.,?!;:\'\"') in
COMMON_WORDS)
        if common_count >= MIN_COMMON_WORDS:
            return True
        if common_count / len(words) >= COMMON_WORD_RATIO:
            return True
        if '?' not in text and 2 <= (sum(len(w) for w in words)/len(words)) <= 8
and len(words) >= 3:
            return True
        return False

def brute_force_morbit(cipher_digits, target=""):
    digits = [str(i) for i in range(1,10)]
    results = []
    tested = 0
    for perm in itertools.permutations(BIGRAMS):
        mapping = {digits[i]: perm[i] for i in range(9)}
        tested += 1
        morse = digits_to_bigram_string(cipher_digits, mapping)
        plaintext = morse_to_plain(morse)
        if target:
            if target.lower() in plaintext.lower():
                results.append((mapping, plaintext, tested))
        else:
            if looks_like_english(plaintext):
                results.append((mapping, plaintext, tested))
        if tested % 50000 == 0:
            print(f"[+] Tested {tested} permutations...")
    return results

def main():
    cipher = ''.join(ch for ch in CIPHERTEXT if ch.isdigit())
    if not cipher:
        print("ERROR: Empty ciphertext.")
        sys.exit(1)
    print(f"[+] Brute-forcing Morbit keys, ciphertext length
{len(cipher)}.")
    matches = brute_force_morbit(cipher, TARGET)
    if not matches:
        print("[-] No plausible plaintexts found.")
    else:
        print(f"\n=== {len(matches)} CANDIDATES FOUND ===\n")
        for idx, (mapping, pt, tested) in enumerate(matches, 1):
            print(f"--- Candidate {idx} (tested {tested} perms) ---")

            # Numeric key
            numeric_key = ' '.join(f"{d}->{mapping[d]}" for d in
sorted(mapping.keys(), key=lambda x:int(x)))
            print("Numeric key mapping:")

```

```
print(numeric_key)

# Alphabetical analogue (A=1, B=2, ..., I=9)
letter_key = ' '.join(f"{chr(64+int(d))}->{mapping[d]}" for d in
sorted(mapping.keys(), key=lambda x:int(x)))
print("Alphabetical analogue key:")
print(letter_key)

print("Plaintext:")
print(pt)
print()

if __name__ == "__main__":
    main()
```

```
### FreqAn1 (useless)
```

Occurency and Frequency Analysis 3-grams			
↑↓	↑↓	↑↓	↑↓
196	3x	2.34%	■
373	2x	1.56%	■
593	2x	1.56%	■
168	2x	1.56%	■
866	2x	1.56%	■
388	2x	1.56%	■
865	2x	1.56%	■
186	2x	1.56%	■
132	2x	1.56%	■
743	2x	1.56%	■
881	2x	1.56%	■
575	2x	1.56%	■
891	2x	1.56%	■
121	2x	1.56%	■
327	2x	1.56%	■
625	2x	1.56%	■
686	2x	1.56%	■
712	2x	1.56%	■
148	2x	1.56%	■
495	2x	1.56%	■
383	2x	1.56%	■
167	2x	1.56%	■
269	1x	0.78%	■
651	1x	0.78%	■
812	1x	0.78%	■
185	1x	0.78%	■

FreqAn2 (useless)

11	11	11	11
18	9%	4.60%	
68	8%	4.17%	
16	8%	4.17%	
12	7%	3.65%	
73	7%	3.65%	
81	6%	3.13%	
62	6%	3.13%	
85	6%	3.13%	
31	5%	2.6%	
65	5%	2.6%	
66	5%	2.6%	
24	5%	2.6%	
61	5%	2.6%	
25	5%	2.6%	
95	5%	2.6%	
32	4%	2.08%	
69	4%	2.08%	
38	4%	2.08%	
33	4%	2.08%	
39	4%	2.08%	
71	4%	2.08%	
86	4%	2.08%	
37	3%	1.56%	
48	3%	1.56%	
51	3%	1.56%	
58	3%	1.56%	
27	3%	1.56%	
14	3%	1.56%	
21	3%	1.56%	
26	3%	1.56%	
84	3%	1.56%	
83	3%	1.56%	
56	3%	1.56%	
42	3%	1.56%	
96	3%	1.56%	
59	2%	1.04%	
19	2%	1.04%	
11	2%	1.04%	
43	2%	1.04%	
75	2%	1.04%	
91	2%	1.04%	
74	2%	1.04%	
17	2%	1.04%	
57	2%	1.04%	
35	2%	1.04%	
93	2%	1.04%	
54	1%	0.52%	
82	1%	0.52%	
92	1%	0.52%	
53	1%	0.52%	
28	1%	0.52%	
13	1%	0.52%	
88	1%	0.52%	
67	1%	0.52%	
52	1%	0.52%	
36	1%	0.52%	
47	1%	0.52%	

List of sources:

- [0] The book - INFORMATION SECURITY Principles and Practice - Mark Stamp
- [1] <https://www.boxentriq.com/code-breaking/vigenere-cipher>
- [2] <https://www.guballa.de/vigenere-solver> - more detailed info
- [3] <https://www.boxentriq.com/code-breaking/cipher-identifier>
- [4] <https://www.cryptool.org/en/cto/ncid/>
- [5] <https://decode.com/ru/cipher>
- [6] <https://chatgpt.com>

[7] www.rapidtables.com

[8] https://www.dcode.fr/cipher-identifier?utm_source=chatgpt.com